

All the programs must be developed with the passing of their possible parameters to the function: `int main (int argc, char * argv [])`. The return values of the primitive calls should be tested and the error messages displayed with `perror`. Error messages to the user will be sent to the standard `stderr` file.

Exercise 1: characteristics of a file `stat ()`, `fstat ()`, `lstat ()`

Write a program which retrieves the characteristics of given files, by calling `int stat ()` functions; `int fstat ()`; and `int lstat ()`; . For a given file, display the following characteristics:

- the inode number,
- the size of the file,
- protection,
- the block size,
- the group ID,
- the number of physical links,
- the number of blocks,
- owner ID,
- the time of the last access,

Displaying a time in a readable format can be accomplished using the `ctime ()` or `localtime` (use the unix manual: `man ctime`) function.

Exercise 2: creating of a file: `open ()`

1. Write a program that creates a read / write file, through the call:
`int open (const char * pathname, int flags, mod_t mode);`
If the file already exists, an error should be returned.
2. What is the error code returned when the file already exists?
3. If the specified mode argument is 755 (`rw-r-xr-x`), is the file created with exactly these permissions? Explain!

Exercise 3: Using a file: `open ()`, `read ()`, `write ()`

- A. Write a program which copies the file given as an argument character by character to the standard output, only if the file given as an argument is a regular file (using the `fstat` function).
- B. Write a program that does the work of the `cp` command in low-level I / O.

Exercise 4: descriptor duplication of a `dup ()` file, `dup2 ()`

- A. Write a program that redirects standard error output to a previously created file, `error_file`. That is, any writing of the form `write (2, ..)`, `fprintf (stderr, ...)` must be done in the file `error_file`.
- B. Write a program that redirects standard output to a file, `output_file`, previously created. That is, all writing of the form `write (1, ..)`, `fprintf (stdout, ...)`, `printf (...)` must be done in the file `output_file`.

Exercise 5: Moving the read / write head of files `lseek ()`

- a) Write a program that creates an empty file. Position the read / write head on the 10,000th byte from the start of the file. Write a character at this position.
- b) How big should the file be? Is this the size that is returned by the `ls -l` command? Have the blocks corresponding to the 9,999 character hole been allocated (use `df`)? The `lseek ()` primitive moves the current offset of a file and returns the new offset.

- c) Write a program which, after a certain number of read / write operations on a file, returns the value of the current offset.

The offset is associated with a file and not with a descriptor. If two descriptors refer to the same file, the modification (by reading / writing or lseek ()) of the offset of the file via one descriptor is "visible" via the other descriptor.

- d) Check the previous statement

Exercise 6:

Summary exercise: Create a name, first name, age structure. Write a file management program (text) with welcome menu: possibility to create the file, to read it, to add a file to it, to search for one.